Operating Systems, Security and Networking
Portfolio Report

Name: Karim Salib
ID: 2301228

Module: KH5003CMD
Lecturer: Dr Nada GabAllah
TA: Eng Youssef Athanassious

Submission Date: 14/4/2026

Table of Contents

I. *Cover Page*

II. *Introduction*

III. *Objectives*

IV. *System Design*

V. *Tasks and Implementation*
- *5.1 Task 1*
- *5.2 Task 2*
- *5.3 Task 3*
- *5.4 Task 4*
- *5.5 Task 5*
- *5.6 Task 6*
- *5.7 Task 7*
- *5.8 Task 8*

VI. *Testing and Result*

VII. *Conclusion*

VIII. *References*

Introduction

This portfolio is for the module *KH5003CMD: Operating Systems, Security, and Networking*. It includes a set of programming tasks completed during the course to help understand how different parts of operating systems and networking work in practice.

There are eight main tasks in total, each focusing on a different topic such as process management, memory handling, file systems, networking, and system security. These tasks include building a simple shell, implementing a multithreaded word count program, simulating paging and deadlock detection, comparing CPU scheduling algorithms, analyzing file systems, creating a network chat application, and working with encrypted passwords.

All tasks are implemented using C++. The aim of this portfolio is not just to complete the coding requirements, but also to understand the logic behind how operating systems manage resources, run processes, and handle communication between systems.

Overall, this work combines both theory and practical programming to give a clearer understanding of the core concepts covered in the module.

Objectives

The main goal of this portfolio is to better understand how operating systems and networking concepts work in real life by building programs instead of only learning theory. Throughout this project, the focus is on developing practical skills in C/C++ while also gaining a clearer understanding of how different system components function.

This includes learning how a shell processes commands, how multithreading allows multiple tasks to run at the same time, and how memory management works through paging systems. It also involves understanding more complex topics such as deadlock detection and CPU scheduling and seeing how different algorithms can affect system performance.

In addition, the project aims to explore file systems by analyzing real directory data, and to understand basic networking concepts by creating a chat application using sockets. Basic security ideas are also covered through working with encrypted passwords.

Overall, the objective is to connect the theory taught in the module with practical implementation, making it easier to understand how operating systems and networks work behind the scenes.

System Design

The system design for this portfolio is based on dividing the work into eight separate tasks, with each task focusing on a specific topic from the module. Instead of building one large system, each program was designed as a small, independent solution to a particular problem related to operating systems, networking, or security.

For the operating systems part, the programs cover areas such as process handling, memory management, scheduling, deadlock detection, and file systems. For example, the shell program is designed to take user input, process built-in commands, and run other programs. The paging and scheduling tasks work as simulations, where input data is used to apply different algorithms and produce results. The deadlock detection task reads system data from a file and checks if a deadlock exists.

Some tasks also focus on concurrency and communication. The multithreaded word count program splits a file into parts and processes them at the same time using threads, then combines the results. The network chat application follows a client-server model, allowing users to send messages over a network using sockets.

Overall, the design is simple and modular. Each task has its own input, processing, and output, which makes the programs easier to build, test, and understand.

5.1 Task 1

I didn't complete this task, but the idea was to build a simple shell program that takes commands from the user and runs them.

Basically, the shell reads what the user types, checks if it's a built-in command like cd or echo, and runs it directly. If it's not, it creates a new process to run the program. It also includes features like redirecting input/output and running commands in the background.

5.2 Task 2

Objective:
The objective of this task was to count how many times each word appears in a text file using multiple threads, and then combine the results into a final word-frequency count.

Approach:
The text file was divided into smaller segments, and each segment was assigned to a separate thread. Each thread processed its own part of the file independently and counted the words. After all threads finished, their results were combined to produce the final output.

Code Snippet:

```
for (int i = 0; i < numThreads; i++) {
    threads.push_back(thread(countWords, segments[i], ref(threadCounts[i])));
}

for (auto& t : threads) {
    t.join();
}
```

Explanation:
Each thread is responsible for counting words in a specific part of the file and storing the results in its own map. This avoids conflicts between threads. Once all threads complete execution, the main program waits using join and then merges all the individual results into one final word count.

Output:

```
hello = 3
world = 1
test = 3
```

Conclusion:
This task demonstrated how multithreading can improve performance by dividing work between multiple threads and combining the results efficiently.

5.3 Task 3

This task was not completed properly due to academic issues, but it was about simulating a paging system using the aging algorithm. The idea is to track page usage over time and decide which page to replace in memory when a page fault occurs. It also involves reading page references from a file and analysing how the number of page frames affects performance.

5.4 Task 4

Objective:

The objective of this task was to implement a program that detects whether a deadlock exists in a system with multiple processes and resources.

Approach:
The program reads input data from a file, including the number of processes, resource types, existing resources, allocation matrix, and request matrix. It then applies the deadlock detection algorithm to check if any processes are stuck.

Code Snippet:

```
for (int i = 0; i < p; i++) {
    if (!finished[i]) {
        bool canRun = true;
        for (int j = 0; j < r; j++) {
            if (R[i][j] > available[j]) {
                canRun = false;
                break;
            }
        }
        if (canRun) {
            for (int j = 0; j < r; j++) {
                available[j] += C[i][j];
            }
            finished[i] = true;
        }
    }
}
```

Explanation:
The algorithm checks each process to see if its requested resources can be satisfied using the available resources. If a process can run, it is marked as finished and its allocated resources are released back into the system. This process repeats until no more processes can proceed. If some processes are still not finished at the end, they are considered deadlocked.

Output:

```
Number of processes: 3
Number of resource types: 2

Deadlock detected!

Deadlocked processes:
P1
P2
```

Conclusion:

This task helped in understanding how deadlocks occur and how systems can detect them using resource allocation and request tracking.
Brief summary.

5.5 Task 5

Objective:
The objective of this task was to simulate different CPU scheduling algorithms (FCFS, SJF, and Round Robin) and compare their performance based on average waiting time.

Approach:
The program takes input for multiple processes, including arrival time and burst time. It then applies three scheduling algorithms: FCFS, SJF, and Round Robin. After running each algorithm, the average waiting time is calculated and compared.

Code Snippet:

```
double fc = fcfs(p);
double sj = sjf(p);
double rr = roundRobin(p, q);

cout << "\nAverage Waiting Times:\n";
cout << "FCFS: " << fc << endl;
cout << "SJF: " << sj << endl;
cout << "Round Robin: " << rr << endl;
```

Explanation:
Each algorithm schedules processes in a different way. FCFS runs processes in the order they arrive, SJF selects the process with the shortest burst time, and Round Robin runs each process for a fixed time (quantum) in cycles. The program calculates the waiting time for each process and then finds the average for comparison.

Output:

```
Average Waiting Times:
FCFS: 5
SJF: 3
Round Robin: 4
```

Conclusion:
This task showed how different scheduling algorithms affect waiting time. It also helped compare which algorithm performs better under different conditions.

5.6 Task 6

Objective:
The objective of this task was to scan a directory and record the sizes of all files, then display the results as a histogram based on a given bin size.

Approach:
The program starts from a given directory and goes through all files inside it. It collects the size of each file and groups them into ranges (bins) based on the bin width provided. After that, it counts how many files fall into each range and displays the result as a histogram.

Code Snippet:

```
for (const auto& file : fs::recursive_directory_iterator(path)) {
    if (fs::is_regular_file(file)) {
        sizes.push_back(fs::file_size(file));
    }
}
```

Explanation:
The program uses the filesystem library to go through all files in a directory. Each file size is stored in a vector. Then, the sizes are grouped into bins (for example: 0–1023, 1024–2047). The program counts how many files are in each bin and prints the results as a simple histogram.

Output:

```
0 - 1023 bytes: ###
1024 - 2047 bytes: #####
2048 - 3071 bytes: ##
```

Conclusion:
This task helped in understanding how file systems work and how file sizes can be analysed using histograms.

5.7 Task 7

Objective:

The objective of this task was to create a simple network chat application that allows communication between a client and a server using socket programming.

Approach:
The application was designed using a client-server model. The server waits for a connection, and the client connects to the server using an IP address and port number. Once connected, both sides can send and receive messages. TCP was used as the communication protocol because it is reliable and ensures that messages are delivered correctly.

Code Snippet:

```
send(clientSocket, message.c_str(), message.length(), 0);
recv(clientSocket, buffer, sizeof(buffer), 0);
```

Explanation:
The program uses sockets to establish communication between two systems. The server creates a socket, binds it to a port, and listens for incoming connections. The client connects to the server using the same port. Once the connection is established, messages can be sent using send() and received using recv . TCP is used to ensure reliable communication without data loss.

Output:

```
Client: Hello
Server: Hi, how are you?
Client: I'm good
```

Conclusion:

This task demonstrated how network communication works using sockets. It also showed how client-server systems exchange messages in real time using TCP.

5.8 Task 8

Objective:
The objective of this task was to implement a simple password encryption system using salt value and repeated processing, like how encryption works in UNIX systems.

Approach:

The program stores a list of passwords and assigns a unique salt value to each one. Each password is then encrypted by applying a simple transformation multiple times (25 rounds). After generating the encrypted passwords, the program allows the user to enter a password and salt to check whether it matches any stored encrypted password.

Code Snippet:

```
string encryptPassword(string password, int salt) {
    string encrypted = password;

    for (int round = 0; round < 25; round++) {
        for (int i = 0; i < encrypted.length(); i++) {
            encrypted[i] = encrypted[i] + (salt % 7) + 1;
        }
    }

    return encrypted;
}
```

Explanation:
The program takes a password and applies a simple encryption method by modifying each character based on a salt value. This process is repeated 25 times to simulate stronger encryption. The salt ensures that even if two users have the same password, their encrypted results will be different. The program then compares the encrypted version of the entered password with stored values to check if it is valid.

Output:

```
Generated Encrypted Passwords:
Password 1: apple123 | Salt: 101 | Encrypted: #######
Password 2: hello456 | Salt: 202 | Encrypted: #######
```

Conclusion:
This task demonstrated how password encryption works using salt and repeated processing. It also showed how encrypted passwords can be used to verify user input securely.

Testing and Results

In this section, the implemented programs were tested using different input values to ensure they work correctly and produce the expected results.
For the multithreaded word count program, a sample text file was used, and the program successfully counted the frequency of words. The results showed correct word counts after combining outputs from all threads.
For the deadlock detection program, different input cases were tested. In one case, the program correctly detected a deadlock and displayed the processes involved, while in another case it showed that no deadlock was present.
For the scheduling algorithms program, multiple sets of processes with different arrival and burst times were used. The program successfully calculated the average waiting time for FCFS, SJF, and Round Robin. The results showed that SJF generally produced lower waiting times, while FCFS produced higher waiting times. The bar chart helped visualize the comparison clearly.
For the file system program, the program was tested on a directory with multiple files. It correctly grouped file sizes into ranges and displayed a histogram showing the distribution.
For the network chat application, the program was tested by running both the server and client. Messages were successfully sent and received between users, showing that the communication over the network was working correctly.
For the encrypted passwords program, the program generated encrypted versions of passwords using salt values and repeated processing. It also successfully verified whether an entered password matched the stored encrypted version.
Overall, the results show that the programs worked as expected and produced correct outputs for the tested cases.

Conclusion

Overall, this portfolio helped me gain a better understanding of how operating systems and networking concepts work in practice, not just in theory. Through the different tasks, I explored topics like multithreading, CPU scheduling, memory management, file systems, networking, and basic security, which made it easier to see how these ideas are applied in real systems.

Each task focused on a different area, helping to build a broader understanding of the module. Some tasks showed how system performance can be analyzed, while others demonstrated how communication between systems works using networking concepts.

Some parts of the coursework were more challenging than others, but they helped improve my problem-solving skills and confidence in C++. Even for tasks that were not fully completed, I was still able to understand the main concepts behind them.

This coursework helped connect what I learned in lectures with practical implementation and gave me a clearer idea of how operating systems manage processes, memory, and communication behind the scenes.

References

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
- Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson.
- GeeksforGeeks. *Operating System and C++ Programming Tutorials*. Available at: https://www.geeksforgeeks.org/
- YouTube. *CPU Scheduling Algorithms Explained*. Available at: https://www.youtube.com/watch?v=EWkQl0n0w5M
- YouTube. *C++ Multithreading Tutorial for Beginners*. Available at: https://www.youtube.com/watch?v=RqE8rGQDrIM
- ChatGPT. *Used for some assistance in explaining concepts and improving report writing*. Available at: https://chat.openai.com/

Github Repository Link : *https://github.com/Thesaleeb/OS-Tasks*